

Architectural Blueprint and Strategic Analysis for an AI-ML Based Intelligent Dead Reckoning System

1. Executive Summary

The vulnerability of Global Navigation Satellite Systems (GNSS) to signal attenuation, multipath interference, and outright denial in urban canyons, tunnels, and dense foliage poses a critical challenge for continuous vehicle tracking. Relying solely on low-cost Micro-Electro-Mechanical Systems (MEMS) Inertial Measurement Units (IMUs) embedded in commercial smartphones during these outages typically results in unbounded cubic positional drift due to the double integration of uncompensated sensor noise and bias instability. Problem Statement SIH 26168, sponsored by the Indian Space Research Organisation (ISRO), addresses this vulnerability by mandating the development of an AI-ML based Intelligent Dead Reckoning (DR) system. This system must provide seamless navigation during GNSS or NavIC (Navigation with Indian Constellation) outages¹, strictly executing on smartphone edge hardware while maintaining positional drift below ten percent of the total distance traveled during the outage³.

This comprehensive report provides an exhaustive architectural, mathematical, and strategic blueprint for resolving this challenge. The analysis systematically deconstructs the problem statement to separate hard constraints from avenues for open innovation. It provides a deep dive into the mandatory IO-VNBD dataset, establishing robust methodologies for training, cross-validation, and benchmarking. Furthermore, the report critically evaluates seven state-of-the-art navigation paradigms, ranging from classical Error-State Kalman Filters (ESKF) to Physics-Informed Neural Networks (PINNs) and deep-learning inertial odometry frameworks. These paradigms are synthesized into five end-to-end architectures, which are rigorously scored against edge feasibility, generalization risk, and hackathon demonstration potential. The analysis concludes with strategic recommendations tailored for a Smart India Hackathon (SIH) environment, providing a phased Minimum Viable Product (MVP) roadmap, hardware and software specifications, and a compelling demonstration strategy designed to validate the system's efficacy in real-time.

2. Deconstruction of Problem Statement SIH 26168

To engineer a solution that not only meets but exceeds the expectations of the ISRO evaluating jury, a rigorous parsing of the problem statement is imperative. The challenge bridges classical inertial navigation physics with modern edge-deployed artificial intelligence, demanding a highly specific set of deliverables that must be clearly categorized.

2.1 Explicit Requirements (Hard Constraints)

The problem statement outlines non-negotiable parameters that serve as the foundational constraints of the proposed system. Foremost is the requirement for smartphone on-device inference, dictating that the system must run entirely on smartphone hardware without relying

on cloud compute during GNSS outages⁴. This constraint severely limits the parameter count and computational complexity of the chosen AI models. Second, the Inertial Odometry Vehicle Navigation Benchmark Dataset (IO-VNBD) is explicitly designated as the mandatory data source for training, validation, and benchmarking⁶.

The architecture must demonstrate GNSS and INS fusion, requiring a mechanism to tightly or loosely couple satellite signals with inertial data during periods of visibility to continuously estimate and correct IMU biases⁹. AI must be explicitly employed for speed and vibration filtering, compensating for the inherently high noise density of smartphone MEMS sensors¹¹. Because a smartphone can be placed in any orientation within a vehicle, an automated algorithm must perform phone alignment and calibration to virtually align the smartphone's arbitrary coordinate frame with the vehicle's body frame¹³.

To bound the accumulated drift, the system must incorporate map matching and Non-Holonomic Constraints (NHC), enforcing physical vehicle kinematics (such as zero lateral velocity) and snapping the estimated trajectory to known road geometries¹⁵. The transition between GNSS-aided navigation and pure inertial dead reckoning must be a seamless GNSS outage and recovery mechanism operating within milliseconds¹⁸. Finally, the ultimate measure of success is the <10% positional-drift benchmark, meaning the accumulated error during pure dead reckoning must not exceed ten percent of the total distance traveled³.

2.2 Implied and Suggested Techniques

Beyond the explicit constraints, the problem statement implies several technical necessities. The requirement for GNSS and INS fusion strongly suggests the implementation of an Error-State Kalman Filter (ESKF) or an Unscented Kalman Filter (UKF). These classical recursive filters are virtually mandatory for managing the covariance matrices of the navigation states and seamlessly accepting asynchronous updates from the GNSS receiver and the AI modules⁹. Furthermore, because AI models trained on specific vehicle dynamics often fail to generalize to new vehicles with different suspension systems or tire pressures, the use of transfer learning or domain adaptation techniques is heavily implied to ensure robustness across diverse hardware¹¹. The requirement for edge deployment also implies the necessity of model quantization techniques, transforming standard floating-point neural networks into 8-bit integer (INT8) formats to satisfy mobile thermal and latency constraints⁴.

2.3 Avenues for Open Innovation

While the constraints dictate the necessary outcomes, the specific architectural mechanisms leave significant room for innovation. The precise role of AI within the pipeline is highly flexible. Artificial intelligence can be utilized strictly to estimate forward velocity from IMU data, or it can dynamically adjust the measurement noise covariance matrices within the Kalman Filter, or it can attempt to replace the entire physics-based integration engine via end-to-end neural networks²².

Similarly, the choice of map matching algorithm is open. Teams may choose between classical Hidden Markov Model (HMM) Viterbi approaches utilizing transition and emission probabilities, or explore Conditional Random Field (CRF) enhanced deep learning approaches for trajectory snapping²⁵. Finally, teams can innovate in their choice of edge runtimes, utilizing frameworks

like LiteRT (formerly TensorFlow Lite) or ONNX Runtime Mobile targeting specific Android Neural Processing Units (NPUs) or Digital Signal Processors (DSPs) to achieve microsecond latency⁴.

3. Exhaustive Analysis of the IO-VNBD Dataset

The IO-VNBD (Inertial Odometry Vehicle Navigation Benchmark Dataset) is a pioneering, large-scale public dataset specifically designed for ground vehicle positioning in GNSS-deprived environments⁶. A rigorous strategy for processing, training, and benchmarking against this dataset is the single most critical factor for success in this SIH challenge.

3.1 Dataset Topology and Sensor Modalities

The IO-VNBD dataset encompasses approximately 98 hours of driving data, covering over 5,700 kilometers across diverse environmental scenarios including urban networks, rural mud roads, and highways, capturing dynamic states such as hard braking and sharp cornering⁶. The dataset is bifurcated into two primary, synchronized subsets. The first subset consists of vehicle extracted data captured directly from the Controller Area Network (CAN) bus of a Ford Fiesta Titanium, totaling roughly 40 hours and 1,300 kilometers. This subset includes high-fidelity ground truth proxies sampled at 10Hz, including individual wheel speeds (front-left, front-right, rear-left, rear-right), steering angle, yaw rate, and indicated longitudinal and lateral accelerations⁶.

The second subset comprises smartphone recorded data, totaling approximately 58 hours and 4,400 kilometers, recorded via Android smartphones securely mounted in four different vehicles: a Ford Fiesta Titanium, Volvo XC70, Renault Megane, and Toyota Corolla Verso⁶. This data includes standard 10Hz MEMS accelerometer, gyroscope, magnetometer, and GNSS receiver outputs⁶. The dataset also includes stationary data specifically intended to aid in the estimation of initial sensor biases⁶.

3.2 Methodologies for Training, Validation, and Benchmarking

Training a robust neural network solely on smartphone IMU data against intermittent GNSS positions is notoriously difficult due to the low sampling rate of GNSS and the lack of high-resolution ground truth for intermediate kinematic states. The optimal training strategy must exploit the multi-modal nature of IO-VNBD through a phased supervised learning regimen.

The initial phase requires supervised pre-training utilizing the CAN-bus data as proxy ground truth. The neural network should be trained using the high-fidelity CAN-bus wheel speeds and steering angles as target variables against the vehicle's IMU inputs. This teaches the network to infer true vehicle dynamics—effectively learning a virtual odometer—from noisy inertial data⁶. Following pre-training, domain adaptation techniques, such as those demonstrated in the Recalibrated Wheel Odometry Neural Network (R-WhONet) architecture, must be applied to transfer the physical dynamics learned from the CAN-bus data to the noisier, lower-grade MEMS sensors of the smartphone dataset¹¹.

Validation must be executed utilizing a strict leave-one-vehicle-out or leave-one-environment-out cross-validation strategy. Because the IO-VNBD dataset classifies drivers and styles (e.g., defensive versus aggressive driving, varying tire pressures), the

validation split must ensure that the model is tested on entirely unseen driving dynamics to prevent overfitting to a specific vehicle's suspension characteristics⁶.

To rigorously benchmark the <10% drift constraint, artificial GNSS outages of 30, 60, 120, and 180 seconds must be programmatically injected into the continuous testing sequences¹². The system will operate in GNSS-aided mode until the simulated outage, at which point it transitions to pure AI dead reckoning. The primary error metric will be the Cumulative True Error (CTE) and Cumulative Root Mean Square Error (CRSE), calculated as the Euclidean distance between the AI-estimated trajectory and the hidden ground-truth GNSS trajectory at the conclusion of the outage¹².

4. Evaluation of Fundamental Navigation Approaches

To construct the optimal system, multiple technically credible approaches blending classical physics with machine learning must be systematically analyzed. The integration of ML with inertial navigation falls onto a spectrum ranging from purely analytical to purely data-driven, each presenting distinct advantages, computational profiles, and failure modes.

4.1 Classical INS + ESKF/UKF + Map Matching

This approach represents the traditional, purely analytical strapdown inertial navigation system (SDINS). Raw IMU data is mathematically integrated to yield position, velocity, and attitude. An Error-State Kalman Filter (ESKF) fuses GNSS data to continuously estimate and correct the IMU biases and attitude errors⁹. During GNSS outages, the system relies entirely on Non-Holonomic Constraints (NHC) acting as pseudo-measurements to bound drift, followed by map matching to snap the trajectory to known roads¹⁵. In this architecture, AI plays no role in the core state estimation. While computational costs are virtually zero and smartphone feasibility is absolute, the accuracy potential during extended outages (>30 seconds) is exceedingly poor.

Uncompensated MEMS random walk noise causes cubic positional drift during double integration. The SIH demo potential is extremely low, as it fails to satisfy the core "AI-ML based" mandate of the problem statement.

4.2 AI Velocity Estimation + INS + ESKF/UKF

This architecture utilizes deep neural networks, such as 1D Convolutional Neural Networks (CNNs), Long Short-Term Memory (LSTM) networks, or Transformers, to process sliding windows of IMU data and directly regress the vehicle's forward velocity²⁹. The AI functions as a virtual wheel speed sensor (pseudo-odometry), entirely replacing physical odometers. This AI-derived velocity is fed into the ESKF as an external measurement alongside standard NHC. This approach requires supervised training mapping IMU windows to GNSS-derived or CAN-derived velocities. The accuracy potential is high; by bypassing the double-integration of raw accelerations, the system avoids cubic drift, reducing the error accumulation to a linear function of time. Smartphone feasibility is excellent, as a 1D CNN for velocity regression requires minimal parameters (often <2MB) and executes in under 5 milliseconds on an Android NPU using INT8 quantization⁴. While the novelty is moderate, the generalization risk is low to medium, provided the network is trained across diverse vehicle types as present in IO-VNBD.

4.3 Deep-Learning Inertial Odometry (TLIO)

Tightly-Coupled Deep-Learning Inertial Odometry (TLIO) represents a shift toward end-to-end learning. Using architectures typically based on ResNet-1D, the neural network processes IMU data to output a 3D displacement vector over a fixed time window, alongside a dynamically calculated covariance matrix representing the network's uncertainty²⁹. This stochastic measurement is then tightly fused into a cloning EKF, which solves for pose, velocity, and sensor biases²⁹. The AI's role is the end-to-end regression of relative 3D pose changes and heteroscedastic uncertainty estimation. The accuracy potential is very high because the EKF can optimally weight the AI predictions against its kinematic model based on the network's self-reported confidence. Smartphone feasibility remains high, though calculating dense covariance matrices adds computational overhead. The primary failure mode is overconfident network predictions, where inaccurate covariance estimation leads to irreversible filter divergence.

4.4 Neural/Learned Kalman Filtering (KalmaNet)

In this highly innovative approach, the system retains the physical state transition matrices of a classical ESKF, thereby preserving kinematic laws. However, the calculation of the Kalman Gain—which traditionally relies on strict, often inaccurate assumptions of Additive White Gaussian Noise (AWGN)—is replaced entirely by a Recurrent Neural Network, such as a GRU or LSTM²⁴. The AI learns the complex, non-linear, and non-Gaussian noise profiles of low-cost MEMS sensors and dynamically outputs the optimal Kalman Gain vector⁹. This requires training the network end-to-end to minimize the mean squared error between the estimated states and the ground truth trajectory. The accuracy potential is exceptional in high-vibration vehicle

environments where classical noise covariance matrices (Q and R) fail to capture true dynamics⁹. The novelty is extremely high, as Model-Based Deep Learning (MBDL) is a cutting-edge research area, though the complexity of backpropagating through a recursive filter structure poses significant training challenges.

4.5 Physics-Informed Neural Networks (PiDR)

Physics-Informed Inertial Dead Reckoning (PiDR) seeks to eliminate the Kalman filter entirely by embedding the rigid differential equations of strapdown inertial navigation directly into the neural network's loss function²². The AI predicts the navigation states, but during the training phase, a custom loss function harshly penalizes predictions that violate kinematic laws (e.g., changes in position must mathematically align with integrated velocities)³³. This approach offers very high generalization potential by preventing the neural network from predicting physically impossible trajectories, a common failure mode in pure deep learning³³. However, the computational cost during training is immense, and balancing the data-driven loss with the physics-driven loss requires extraordinarily delicate hyperparameter tuning.

4.6 AI-Based IMU Bias/Error Correction

This approach represents a conservative, pre-filtering strategy. Instead of predicting kinematic states like position or velocity, a highly lightweight RNN or CNN processes raw IMU data alongside thermal readings to predict the instantaneous, time-varying biases ($\delta b_a, \delta b_g$) of the sensors¹⁰. These highly precise, AI-cleaned signals are then fed into a standard, unmodified

ESKF. The data requirements involve learning the residual errors between raw IMU data and high-end reference IMU data. While computationally trivial and highly feasible for low-end smartphones, the accuracy potential is only moderate. Because the system still relies on the double integration of the cleaned acceleration signals, it remains susceptible to rapid drift during outages exceeding 60 seconds.

4.7 AI-Based GNSS+INS Fusion

This methodology utilizes machine learning not to process the IMU, but to intelligently manage the GNSS/NavIC data prior to an outage. Neural networks evaluate the signal-to-noise ratio, satellite geometry, and multipath interference to dynamically assign reliability weights to the GNSS measurements before they enter the ESKF. During periods of partial GNSS degradation (e.g., driving under heavy foliage rather than a complete tunnel), the AI prevents degraded satellite data from corrupting the IMU bias estimates within the filter. While highly effective at maintaining a pristine filter state just prior to an outage, this approach provides no active assistance during a total GNSS denial event, making it insufficient as a standalone solution for the SIH problem statement.

4.8 Comparative Synthesis of Navigation Approaches

Approach	AI Role	Accuracy Potential	Edge Feasibility	Generalization Risk	SIH Demo Potential
Classical INS + EKF	None	Low (>30s outages)	Very High	Low	Low (Fails PS mandate)
AI Velocity Estimation	Virtual Odometer	High (Linear drift)	Very High (<5ms)	Medium	High
Deep-Learning IO (TLIO)	Pose + Covariance	Very High	High	High (Domain shifts)	Very High
Neural Kalman Filter	Kalman Gain Calc	Very High	Medium	Medium	Exceptional
Physics-Informed NN	End-to-end States	High	Low (Complex)	Low	Exceptional
AI IMU Bias Correction	Sensor Pre-filtering	Medium	Very High	Low	Medium (Safe

					fallback)
AI GNSS+INS Fusion	Measurement Weighting	Low (During outage)	High	Medium	Low

5. End-to-End Architectural Designs and Scoring

To translate the theoretical approaches into actionable engineering blueprints for the Smart India Hackathon, we synthesize the aforementioned methodologies into five complete end-to-end architectures. These architectures are designed to strictly adhere to the IO-VNBD dataset constraints and the smartphone edge deployment mandate.

5.1 Architecture A: The Neural Virtual Odometer (Safest)

This architecture is designed for maximum reliability and ease of deployment.

- **Data Flow:** The smartphone IMU data undergoes AI-based Virtual Alignment to rotate the signals into the vehicle frame. The aligned signals are fed into a quantized 1D CNN (trained via transfer learning on IO-VNBD CAN-bus data) to predict forward velocity. This velocity, alongside mathematical Non-Holonomic Constraints, serves as the update measurement for a standard 15-state ESKF. Finally, the estimated trajectory is passed through an HMM Viterbi map matching algorithm utilizing Valhalla to snap to the road network.
- **Rationale:** By converting the IMU into a synthetic wheel speed sensor, the system side-steps the catastrophic cubic drift associated with double integration. It is mathematically stable, highly robust to varying smartphone hardware, and easily deployable via LiteRT.

5.2 Architecture B: The MBDL Neural Filter (Most Innovative)

This architecture represents the cutting edge of Model-Based Deep Learning, targeting the fundamental flaw of classical filtering.

- **Data Flow:** Raw IMU data and intermittent GNSS data undergo Virtual Alignment and populate the state transition matrices of a kinematic model. Instead of calculating the Kalman Gain via standard Riccati equations, a GRU-based Neural Network (KalmaNet) processes the measurement innovations and current state to directly output the Kalman Gain matrix. The updated states are then passed to the map matching module.
- **Rationale:** Low-cost smartphone MEMS sensors exhibit severe non-Gaussian noise and vibration profiles that violate the core assumptions of an ESKF⁹. The neural network dynamically adapts to these noise profiles, providing vastly superior bias estimation just prior to GNSS outages.

5.3 Architecture C: Physics-Informed TLIO (The Optimal Hybrid)

This architecture balances the raw predictive power of deep learning with the strict boundaries of physical kinematics.

- **Data Flow:** A ResNet-1D architecture processes windows of IMU data to output a 3D displacement vector and a heteroscedastic covariance matrix. This output is fed into a stochastic cloning ESKF, which enforces rigid kinematic constraints and integrates the displacements. The output is refined via offline map matching.
- **Rationale:** This architecture leverages the neural network's ability to extract complex motion features while using the ESKF to ensure the predicted trajectory remains physically viable. The dynamic covariance output prevents the filter from trusting erroneous neural predictions during anomalous vehicle maneuvers²⁹.

5.4 Architecture D: The Cascaded AI Pre-Filter

This architecture prioritizes ultra-low computational overhead, acting primarily as a signal cleaner.

- **Data Flow:** Raw IMU data is passed through a highly optimized, INT8-quantized Temporal Convolutional Network (TCN) that continuously predicts and subtracts the sensor bias and thermal drift. The pristine IMU signal is then processed by a classical ESKF with strict NHC, followed by CRF-enhanced map matching.
- **Rationale:** This is the ultimate fallback architecture. If complex end-to-end neural networks fail to train or generalize to new vehicles, signal denoising is a guaranteed, mathematically safe method to extend the viability of pure dead reckoning.

5.5 Architecture E: The End-to-End PINN Engine

This architecture abandons the Kalman filter entirely in favor of a monolithic neural structure bounded by physical loss functions.

- **Data Flow:** A large-scale Transformer model processes IMU sequences and directly outputs global coordinates (Latitude/Longitude). During training on the IO-VNBD dataset, a Physics-Informed loss function penalizes the network if its coordinate predictions imply velocities or accelerations that contradict the raw IMU inputs³³.
- **Rationale:** While mathematically elegant and highly resistant to non-linear noise, the computational demands of a Transformer and the immense difficulty of tuning the physics-loss hyperparameters make this a high-risk, high-reward academic pursuit rather than a stable production system.

5.6 Comparative Architectural Scoring Matrix

The architectures are scored based on metrics critical to success in the SIH environment, utilizing a weighted 100-point scale.

Metric (Weight)	Arch A: Virtual Odo	Arch B: Neural Filter	Arch C: Physics TLIO	Arch D: AI Pre-Filter	Arch E: PINN Engine
Drift < 10% (30%)	27 / 30	25 / 30	28 / 30	21 / 30	26 / 30
Edge	18 / 20	14 / 20	15 / 20	19 / 20	10 / 20

Feasibility (20%)					
Innovation (20%)	12 / 20	19 / 20	18 / 20	10 / 20	20 / 20
Generaliza tion (15%)	13 / 15	10 / 15	12 / 15	14 / 15	11 / 15
Demo Reliability (15%)	14 / 15	11 / 15	12 / 15	14 / 15	8 / 15
Total Score	84 / 100	79 / 100	85 / 100	78 / 100	75 / 100

6. Core Sub-System Mathematical Formulation

Regardless of the high-level architecture chosen for the SIH competition, three physical sub-systems must be meticulously engineered and mathematically formulated to satisfy ISRO's requirements for alignment, constraints, and map matching.

6.1 The Error-State Kalman Filter (ESKF) with Non-Holonomic Constraints

The core navigation engine relies on a 15-dimensional error state vector, defined to track the perturbations in the navigation equations. The state vector is expressed as:

$$\delta \mathbf{x} = [\delta \mathbf{p}^n, \delta \mathbf{v}^n, \phi^n, \delta \mathbf{b}_a^b, \delta \mathbf{b}_g^b]^T$$

Where $\delta \mathbf{p}^n$ represents the position error, $\delta \mathbf{v}^n$ is the velocity error, ϕ^n denotes the attitude error (misalignment angles), and $\delta \mathbf{b}_a^b, \delta \mathbf{b}_g^b$ are the accelerometer and gyroscope biases in the body frame, respectively⁹. The state transition matrix propagates these errors based on the specific force and Earth rotation parameters⁹.

During a GNSS outage, the system must rely on Non-Holonomic Constraints (NHC). The fundamental assumption of NHC is that a standard ground vehicle does not move laterally (slip) or vertically (jump) under normal driving conditions³⁵. Let the true velocity of the vehicle in its

body frame (b -frame) be denoted as $\mathbf{v}^b = [v_x^b, v_y^b, v_z^b]^T$. The constraints dictate that:

$$v_y^b \approx 0 + \nu_y, \quad v_z^b \approx 0 + \nu_z$$

where ν_y, ν_z represent zero-mean Gaussian measurement noise, effectively modeling minor slippage and suspension vibrations³⁶.

To apply this to the ESKF, the velocity computed by the INS in the navigation frame ($\mathbf{v}^n$) is rotated into the body frame using the Direction Cosine Matrix ($\mathbf{C}_n^b$). The measurement equation for the ESKF during GNSS denial is formulated as:

$$\mathbf{z}_{NHC} = \mathbf{C}_n^b \mathbf{v}^n - \begin{bmatrix} v_x^b \\ 0 \\ 0 \end{bmatrix} = \mathbf{H} \delta \mathbf{x} + \mathbf{n}_{NHC}$$

Fusing this constraint as a continuous measurement update drastically reduces lateral and vertical drift. Consequently, the remaining positional error is forced almost entirely into the along-track (forward) direction, which is then mitigated by the AI velocity estimator¹⁶.

6.2 AI-Based Virtual Alignment (Smartphone to Vehicle Frame)

Because a smartphone can be mounted arbitrarily on a dashboard or center console, the raw IMU readings exist in the smartphone's coordinate frame (s -frame). These must be mathematically rotated to align with the vehicle's body frame (b -frame) prior to integration¹³.

This virtual alignment process requires determining the Direction Cosine Matrix $\mathbf{C}_s^b$ without physical rotation³⁹.

1. **Vertical Alignment (Pitch and Roll):** When the vehicle is stationary or moving at a constant velocity, the accelerometer exclusively measures the local gravity vector $\mathbf{g}^s$. The pitch and roll angles required to align the smartphone's Z-axis with the gravity vector are deterministically calculated⁴⁰.
2. **Forward Alignment (Yaw):** Determining the vehicle's forward direction is more complex. By taking the derivative of the GNSS velocity prior to the outage, or by applying a low-pass filter to isolate the longitudinal acceleration during the vehicle's initial acceleration phase, a forward vector $\mathbf{a}_{fwd}^s$ is established. The yaw angle required to align the smartphone's Y-axis with $\mathbf{a}_{fwd}^s$ completes the DCM¹³.

To satisfy the "AI-ML based" requirement, a lightweight Multi-Layer Perceptron (MLP) can be trained to continuously monitor this alignment based on the covariance of lateral and longitudinal accelerations. If the phone shifts during transit (e.g., falling off a mount), the AI detects the anomalous variance shift and dynamically recalculates $\mathbf{C}_s^b$ on the fly¹⁴.

6.3 Hidden Markov Model (HMM) Viterbi Map Matching

Even with AI velocity estimation and strict ESKF constraints, longitudinal drift will accumulate over time. Map matching utilizes road network topologies to snap the divergent trajectory back to physical, drivable roads⁴³. Open-source routing engines like Valhalla or OSRM, which can be cross-compiled to C++ for offline Android execution, provide the underlying road geometries⁴³. The map matching process is formulated as a Hidden Markov Model (HMM). In this model, the hidden states are the candidate road segments on the map, and the observations are the noisy positions estimated by the AI-DR system¹⁷.

- **Emission Probability** ($P(o_t|r_i)$): This represents the probability of observing the estimated position o_t given that the vehicle is actually on road segment r_i . It is modeled as a Gaussian distribution based on the perpendicular distance from the estimated point to the road segment:

$$P(o_t|r_i) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{\text{dist}(o_t, r_i)^2}{2\sigma^2}\right)$$

Crucially, the standard deviation σ is not static; it scales dynamically with the growing covariance matrix outputted by the ESKF, widening the search radius as the outage continues¹⁷.

- **Transition Probability** ($P(r_i \rightarrow r_j)$): This represents the probability of moving from road segment r_i to r_j . It penalizes topologically impossible maneuvers and favors the most direct routes:

$$P(r_i \rightarrow r_j) \propto \exp\left(-\frac{|\text{dist}_{\text{network}}(r_i, r_j) - \text{dist}_{\text{Euclidean}}(o_{t-1}, o_t)|}{\beta}\right)$$

where β is an empirical constant balancing the likelihood of route deviations²⁶.

The **Viterbi Algorithm** utilizes dynamic programming to recursively calculate the most probable sequence of road segments by maximizing the joint probability of emissions and transitions across a sliding temporal window⁴⁵. This effectively snaps the trajectory to the road network, correcting accumulated lateral and longitudinal drift.

7. Edge Deployment on Smartphone Hardware

Executing continuous navigation algorithms on an Android device demands stringent software optimization to prevent unacceptable battery drain and thermal throttling, which can drastically reduce processor clock speeds and ruin inference latency²¹.

7.1 On-Device Inference Frameworks and Android HAL

To guarantee execution during total connectivity loss, the AI models must be entirely bundled

onto the device.

- **LiteRT (TensorFlow Lite)** is the optimal framework for this stack. It facilitates the conversion of complex PyTorch or Keras models into highly compressed flat-buffers, typically under 2MB, minimizing the application's storage footprint⁴.
- **ONNX Runtime Mobile** offers a robust alternative. By utilizing the Android Neural Networks API (NNAPI), ONNX Runtime can intelligently delegate matrix operations to the device's NPU, GPU, or CPU based on real-time availability and thermal thresholds⁴.

To acquire data with minimal latency, the application must interface directly with the Android Hardware Abstraction Layer (HAL). Developers must utilize `Sensor.TYPE_ACCELEROMETER` and `Sensor.TYPE_ROTATION_VECTOR`, specifically configuring the sensors to utilize hardware FIFO (First In, First Out) queues⁴⁹. Properly managing wake-up versus non-wake-up sensor FIFOs ensures that the Application Processor (AP) is not needlessly interrupted, conserving battery life while guaranteeing a continuous 10Hz to 100Hz data stream⁴⁹.

7.2 Quantization and Thermal Management

Deploying models utilizing double-precision (FP64) or single-precision (FP32) weights will cause rapid thermal throttling if run continuously at 100Hz on a mobile CPU, leading to dropped frames and missed IMU readings²¹.

- **Post-Training Quantization (PTQ)** must be applied to compress the trained models to 8-bit integers (INT8). Research demonstrates that INT8 quantization of 1D CNNs for IMU time-series data results in less than a 2% degradation in accuracy while reducing memory footprint by 75% and accelerating inference to sub-10 milliseconds⁴.
- By executing this highly compressed INT8 model on a dedicated NPU (e.g., the Qualcomm Hexagon DSP found in Snapdragon processors), power consumption is reduced to milliwatt levels. This hardware-software co-design ensures the dead reckoning navigation service can run uninterrupted in the background without degrading the user experience²¹.

8. Strategic Recommendations for SIH 2026

Based on the preceding technical and architectural analysis, the following structural decisions are definitively recommended for a team competing in the Smart India Hackathon to secure a victory under ISRO's evaluation criteria.

8.1 Best Overall Architecture: Architecture C (Physics-Informed TLIO)

For a team possessing strong fundamentals in deep learning and probabilistic robotics, combining a ResNet-1D displacement estimator with a covariance-cloning ESKF offers the absolute highest accuracy ceiling²⁹. It intrinsically satisfies the requirement for AI-driven noise filtering while maintaining the rigorous, proven mathematical guarantees of classical kinematics.

8.2 Safest & Best for College Teams: Architecture A (Virtual Odometer)

For the majority of undergraduate teams, attempting to debug a tightly coupled Neural EKF during a 36-hour hackathon is an excessively high-risk endeavor. A loosely coupled approach

where a simple, easily trained 1D CNN converts IMU data to a 1D forward velocity (acting as an artificial wheel-speed sensor) is highly achievable. It is easily trainable using supervised learning on the IO-VNBD dataset and is incredibly robust in real-world edge deployment¹¹.

8.3 Most Innovative: Neural Kalman Filter (KalmaNet)

Replacing the ESKF's mathematically rigid Kalman Gain matrix with a Recurrent Neural Network represents the bleeding edge of Model-Based Deep Learning²⁴. An ISRO jury of aerospace engineers will be highly impressed by an architecture that explicitly acknowledges the theoretical limitations of Additive White Gaussian Noise in classical Kalman theory and solves it via data-driven dynamic weighting.

8.4 Best Fallback Strategy: Pure INS + Map Matching

If all neural network training fails prior to the hackathon finale, the team must immediately pivot to deploying a classic 15-state ESKF augmented with aggressive Non-Holonomic Constraints and robust Valhalla-based HMM map matching¹⁰. While it lacks the required AI component and will likely suffer point deductions, it guarantees a functional, demonstrable offline routing system that will outperform teams whose complex AI models crash on the device.

9. Implementation Roadmap, Risks, and SIH Demo Strategy

Success at SIH 2026 requires more than a sound theoretical architecture; it requires translating complex mathematics into a flawless, visually compelling demonstration under extreme time pressure.

9.1 MVP to Final Product Roadmap

- **Phase 1: Dataset Mastery & Pipeline Construction (Weeks 1-2)**
 - Ingest the 98 hours of IO-VNBD data. Build Python data loaders to extract the 10Hz CAN-bus wheel speeds and synchronize them perfectly with the smartphone IMU data to create supervised training pairs⁶. Establish the cross-validation splits, ensuring the Ford Fiesta and Volvo XC70 data are kept separate to test generalization⁷.
- **Phase 2: Core Algorithm Development (Weeks 3-5)**
 - Develop the AI Virtual Alignment algorithm to mathematically rotate the s -frame to the b -frame. Train the PyTorch models for the chosen architecture (e.g., Architecture A's 1D CNN Velocity Estimator). Programmatically inject 30, 60, and 120-second GNSS outages into the validation sets and mathematically prove offline that the drift remains below 10%¹².
- **Phase 3: Edge Deployment & Android Integration (Weeks 6-7)**
 - Apply Post-Training Quantization to convert the PyTorch models to INT8 LiteRT flat-buffers⁴. Build a native Android application utilizing the android.sensor API with hardware FIFOs to ensure low-latency data acquisition without thermal throttling²¹.
- **Phase 4: Map Matching & UI Refinement (Week 8)**
 - Integrate the Valhalla offline routing engine via Java Native Interface (JNI). Implement the

HMM Viterbi snapping algorithm to fuse the raw AI-DR trajectory with the OpenStreetMap road geometries⁴³.

9.2 Critical Experiments and Risk Mitigation

Prior to the competition, the team must empirically validate the system against major failure modes:

1. **Vibration Tolerance Testing:** Test the Android application with the smartphone mounted rigidly to a dashboard versus resting loosely in a cupholder. The virtual alignment algorithm must rapidly re-converge if the phone shifts¹⁴.
2. **Filter Divergence Simulation:** Intentionally feed the ESKF wildly inaccurate AI velocity predictions to ensure the covariance matrices properly reject the outliers without crashing the application.
3. **Outage Duration Scaling:** Plot Positional Error (%) against Outage Time (0-180 seconds). The team must present undeniable empirical evidence that the drift scales linearly (due to AI velocity estimation) rather than cubically (due to raw double integration), remaining safely below the 10% benchmark¹².

9.3 Compelling SIH Demo Strategy

The ISRO judges must witness the system seamlessly transition from GNSS reliance to pure, highly accurate AI-driven dead reckoning in real-time.

- **The Physical Setup:** Build an Android application featuring a split-screen map display. One line represents the raw, uncorrected GNSS coordinate feed; the other represents the AI-DR estimated trajectory. Mount the smartphone securely on a remote-control (RC) car or, if permissible, a full-sized physical vehicle.
- **The Execution Phase:** While navigating outdoors in clear skies, both the GNSS track and the AI-DR track will overlap perfectly as the ESKF continuously estimates biases.
- **The "Faraday" Moment:** To simulate a sudden, total GNSS outage (e.g., entering a tunnel), radically obscure the phone's GNSS receiver by abruptly wrapping a signal-blocking Faraday cage over the antenna, or driving under a dense concrete parking structure.
- **The Climax:** The raw GNSS track on the UI will immediately freeze or jump wildly due to multipath errors. In stark contrast, the AI-DR track must seamlessly detach from its GNSS dependency, smoothly and accurately progressing along the map based solely on the INT8 neural network inference, Non-Holonomic Constraints, and Viterbi map matching. Concluding the drive and removing the Faraday cage should reveal the AI-DR terminal position is within 10% of the true distance traveled, securing a definitive victory.

Works cited

1. Glossary - SBG Systems, <https://www.sbg-systems.com/glossary/>
2. Integration of Low-Cost IMU with MEMS and NavIC/IRNSS Receiver, https://www.researchgate.net/publication/355242571_Integration_of_Low-Cost_IMU_with_MEMS_and_NavICIRNSS_Receiver_for_Land_Vehicle_Navigation
3. SIH Buddy: SIH 2026 Problem Statements — All 226, Scored,

- <https://sih-buddy.vercel.app/>
4. Edge Intelligence Unplugged: Crafting Offline-AI IoT Apps ... - A-bots,
<https://a-bots.com/blog/Offline-AI-IoT-Apps>
 5. Aman's AI Journal • Primers • ML Runtimes,
<https://aman.ai/primers/ai/ml-runtimes/>
 6. IO-VNBD: Inertial and Odometry Benchmark Dataset for Ground,
<https://arxiv.org/pdf/2005.01701>
 7. (PDF) IO-VNBD: Inertial and Odometry Benchmark Dataset for,
https://www.researchgate.net/publication/341149322_IO-VNBD_Inertial_and_Odometry_Benchmark_Dataset_for_Ground_Vehicle_Positioning
 8. IO-VNBD: Inertial and Odometry Benchmark Dataset for Ground,
<https://arxiv.org/abs/2005.01701>
 9. (PDF) Deep Learning Based Kalman Filter for GNSS/INS Integration,
https://www.researchgate.net/publication/371591207_Deep_Learning_Based_Kalman_Filter_for_GNSSINS_Integration_Neural_Network_Architecture_and_Feature_Selection
 10. Gaussian–Student's t Mixture Distribution-Based Robust Kalman,
<https://www.mdpi.com/2072-4292/16/24/4716>
 11. R-WhONet: recalibrated wheel odometry neural network for,
https://www.researchgate.net/publication/388147825_R-WhONet_recalibrated_wheel_odometry_neural_network_for_vehicular_positioning_using_transfer_learning
 12. Wheel Odometry neural Network for vehicular localisation in GNSS,
https://www.researchgate.net/publication/354353720_WhONet_Wheel_Odometry_neural_Network_for_vehicular_localisation_in_GNSS-deprived_environments
 13. Advanced Navigation System for Aircraft Applications,
<https://www.publicationsdrdo.in/index.php/dsj/article/download/4254/2502/12250>
 14. Application of MEMS Sensors for Evaluation of the Dynamics ... - MDPI,
<https://www.mdpi.com/1424-8220/21/8/2881>
 15. Dual-Branch INS/GNSS Fusion with Inequality and Equality ... - arXiv,
<https://arxiv.org/html/2602.21266v1>
 16. (PDF) Observability Analysis of Non-Holonomic Constraints for Land,
https://www.researchgate.net/publication/278245054_Observability_Analysis_of_Non-Holonomic_Constraints_for_Land-Vehicle_Navigation_Systems
 17. Advancing Map-Matching and Route Prediction - MDPI,
<https://www.mdpi.com/2079-9292/14/18/3608>
 18. Official Problem Bank - Amity Internal Hackathon,
<https://amityhackathon.vercel.app/problems?theme=miscellaneous>
 19. An Enhanced, Real-Time, Low-Cost GNSS/INS Integrated ... - PMC,
<https://pmc.ncbi.nlm.nih.gov/articles/PMC11991159/>
 20. An Improved Multi-State Constraint Kalman Filter for Visual-Inertial,
<https://arxiv.org/html/2210.08117v2>
 21. Thermal-Aware Scheduling for Deep Learning on Mobile Devices,
https://www.researchgate.net/publication/379297343_Thermal-Aware_Scheduling_for_Deep_Learning_on_Mobile_Devices_With_NPU

22. Physics-Informed Inertial Dead Reckoning for Autonomous Platforms,
<https://arxiv.org/html/2601.03040v2>
23. Arup Kumar Sahoo - Hatter Postdoctoral Researcher - ResearchGate,
<https://www.researchgate.net/profile/Arup-Sahoo>
24. Model-Based and Data-Driven Kalman filtering,
<https://pergamos.lib.uoa.gr/uoa/dl/object/3422075/file.pdf>
25. Enhanced Map-Matching Algorithm with a Hidden Markov Model for,
https://www.researchgate.net/publication/320721676_Enhanced_Map-Matching_Algorithm_with_a_Hidden_Markov_Model_for_Mobile_Phone_Positioning
26. A Novel Map Matching Method Based on Improved Hidden Markov,
<https://www.tandfonline.com/doi/full/10.1080/17538947.2024.2328366>
27. Learning to Localise Automated Vehicles in Challenging ... - MDPI,
<https://www.mdpi.com/2076-3417/11/3/1270>
28. R-WhONet: Recalibrated Wheel Odometry Neural Network ... - arXiv,
<https://arxiv.org/pdf/2209.05877>
29. (PDF) TLIO: Tight Learned Inertial Odometry - ResearchGate,
https://www.researchgate.net/publication/342758907_TLIO_Tight_Learned_Inertia_I_Odometry
30. A Comparison of Three Measurement Models for the Wheel,
https://www.researchgate.net/publication/353546500_A_Comparison_of_Three_Measurement_Models_for_the_Wheel-Mounted_MEMS_IMU-Based_Dead_Reckoning_System
31. Wheel-AINS: A Vehicle Autonomous Positioning System Based on a,
<https://pmc.ncbi.nlm.nih.gov/articles/PMC13413940/>
32. On-Device AI on Android in 2026: What Gemma 4 Means for Your,
<https://medium.com/@njiang.pin/on-device-ai-on-android-in-2026-what-gemma-4-means-for-your-tflite-classifiers-ffb349e9e28f>
33. The prediction results of states $[ax, vx, ay, vy]$ and sensor,
https://www.researchgate.net/figure/The-prediction-results-of-states-ax-vx-ay-vy-and-sensor-measurements_fig8_373092081
34. (PDF) A-PINN: Auxiliary Physics-informed Neural Networks for,
https://www.researchgate.net/publication/399476869_A-PINN_Auxiliary_Physics-informed_Neural_Networks_for_Structural_Vibration_Analysis_in_Continuous_Euler-Bernoulli_Beam
35. An improved Kalman filter algorithm for tightly GNSS/INS integrated,
<https://www.aimspress.com/article/doi/10.3934/mbe.2024040?viewType=HTML>
36. Techniques in Kalman Filtering for Autonomous Vehicle Navigation,
https://vtechworks.lib.vt.edu/bitstream/handle/10919/78128/etd-05192015-184107_Jones_PA_T_2015.pdf?isAllowed=y&sequence=1
37. An Adaptive Combined Filtering Algorithm for Non-Holonomic,
<https://www.mdpi.com/2072-4292/17/7/1126>
38. Optical and Mass Flow Sensors for Aiding Vehicle Navigation in,
<https://www.mdpi.com/1424-8220/20/22/6567>
39. Advanced Navigation System for Aircraft Applications - ResearchGate,
https://www.researchgate.net/publication/270491929_Advanced_Navigation_System

[em_for_Aircraft_Applications](#)

40. Improving the Accuracy of Human Body Orientation Estimation With,
https://www.researchgate.net/publication/312180455_Improving_the_Accuracy_of_Human_Body_Orientation_Estimation_With_Wearable_IMU_Sensors
41. Measuring Orientation of Human Body Segments Using Miniature,
https://www.researchgate.net/publication/7873311_Measuring_Orientation_of_Human_Body_Segments_Using_Miniature_Gyroscopes_and_Accelerometers
42. mtl305e user manual - Mouser Electronics,
<https://www.mouser.com/pdfDocs/7430-3316-01MTLT305EUserManual.pdf>
43. Valhalla for Unraid - Community Apps,
<https://ca.unraid.net/apps/valhalla-14d0ffh1cfdtm1>
44. valhalla/valhalla: Open Source Routing Engine for OpenStreetMap,
<https://github.com/valhalla/valhalla>
45. Map Matching: The Viterbi Snapper | by Or Nachmias | Gett Tech,
<https://medium.com/gett-engineering/map-matching-the-viterbi-snapper-e32d11f0d130>
46. An Enhanced Hidden Markov Model for Map-Matching in Pedestrian,
<https://www.mdpi.com/2079-9292/13/9/1685>
47. Research on vehicle trajectory matching method based on improved,
<https://www.sciopen.com/article/10.62756/jmsi.1674-8042.2024024>
48. Smart Glasses Technology Explained: Complete Architecture Guide,
<https://banna-tech.com/smart-glasses-technology-explained/>
49. sensors/1.0/types.hal - platform/hardware/interfaces - Git at Google,
<https://android.googlesource.com/platform/hardware/interfaces/+/main/sensors/1.0/types.hal>
50. CPU vs GPU vs TPU vs NPU: AI Hardware Architecture Guide 2026,
<https://www.eigenstate.dev/essay/cpu-vs-gpu-vs-tpu-vs-npu-ai-hardware-architecture-guide-2026>